

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Torneo de Sudoku:
Validación y Análisis de Partidas
(LFP Numerix)

PONDERACIÓN: 15 pts

Tiempo estimado: 12 hrs

Índice

1. MARCO FORMATIVO.....	3
1.1. Valor.....	3
1.2. Competencia(s).....	3
1.3. Objetivo SMART	3
2. Enunciado de la Práctica.....	5
2.1 Descripción del problema a resolver	8
2.2 Alcance de la práctica.....	8
2.3 Requerimientos técnicos.....	8
3. Entregables	9
4. Material de apoyo	10
5. Recursos y herramientas a utilizar.....	10
6. Cronograma.....	10
7. Rúbrica de Calificación	12
7.1 Requisitos para optar a la calificación.....	12
7.2 Resumen de Puntuaciones	12
7.3 Comentarios Generales.....	14
Detalle de la Calificación	15

1. MARCO FORMATIVO

1.1. Valor

Originalidad del Trabajo	Cada estudiante debe desarrollar su propio código y/o documentación, aplicando los conocimientos adquiridos en el curso.
---------------------------------	--

1.2. Competencia(s)

Tipo de Competencia	
Competencia General	El estudiante debe demostrar una comprensión cabal del lenguaje de programación mediante su aplicación en la resolución de problemas inherentes a las fases de un compilador.
Competencia Específica	El estudiante debe ser capaz de implementar en Python, aplicando Programación Orientada a Objetos, un motor de validación de tableros mediante matrices, con lectura y manipulación de archivos de texto, estructuras de control y colecciones, y el cálculo de métricas de desempeño para analizar los resultados de un torneo de Sudoku.

1.3. Objetivo SMART

SMART	Definición	Objetivo redactado
Específico (¿Qué?)	El objetivo es concreto y tangible.	Los estudiantes serán capaces de desarrollar en Python un programa orientado a objetos que lea archivos de texto delimitados por comas (split(',')) con tableros de Sudoku, jugadores e intentos de resolución, valide las reglas del Sudoku mediante matrices, calcule métricas de desempeño (tiempo promedio y porcentaje de validez) y genere 3 reportes analíticos en formato HTML.
Medible (¿Cuánto?)	El objetivo tiene una medida objetiva de éxito.	Lograr que al menos el 85% de las funcionalidades requeridas (lectura de archivos, validación de tableros y generación de reportes) sean implementadas correctamente y validadas en ejecución.
Alcanzable (¿Cómo?)	El objetivo debe ser posible con los recursos disponibles.	Utilizando únicamente el lenguaje Python (versión 3.x) y las herramientas de desarrollo disponibles en el laboratorio (IDE, consola, documentación oficial).
Realista (¿Para qué?)	El objetivo contribuye a metas más amplias.	Fortalecer las habilidades prácticas en manejo de matrices, validación de reglas, procesamiento de archivos y análisis estadístico de resultados, preparando a los estudiantes para el desarrollo de motores de validación, juegos de lógica y sistemas de evaluación automática.
A Tiempo (¿Cuándo?)	El objetivo tiene fecha límite o mejor	Cumplir este aprendizaje dentro del período establecido en el cronograma académico o en un plazo máximo de 12 horas de desarrollo efectivo.



	aún un cronograma de hitos de progreso.	
--	--	--



2. Enunciado de la Práctica

La plataforma educativa guatemalteca “Numerix Academy” organiza torneos en línea de Sudoku para sus estudiantes de lógica matemática. Cada jugador resuelve tableros de distinta dificultad y sus intentos quedan registrados para su posterior análisis.

Numerix Academy le ha solicitado desarrollar, en lenguaje Python, el motor de calificación y análisis del torneo: un programa de consola capaz de validar si un tablero enviado por un jugador cumple las reglas del Sudoku, calificar su desempeño y generar reportes estadísticos sobre el comportamiento de los jugadores y la dificultad real de los tableros.

A diferencia de un simple registro de resultados, el sistema deberá razonar sobre la estructura del tablero (filas, columnas y cajas de 3x3) para determinar qué tan “correcto” es un intento, incluso cuando el jugador no haya completado el tablero de forma perfecta.

Archivos de Entrada

sudokus.lfp

Este archivo contiene los tableros de Sudoku disponibles en el torneo.

Formato:

```
id_sudoku,dificultad,tablero
```

Donde:

- id_sudoku: Identificador único del tablero (número entero)
- dificultad: Nivel declarado del tablero (Facil, Media, Dificil, Experto)
- tablero: cadena de 81 caracteres (dígitos), recorriendo el tablero fila por fila de izquierda a derecha y de arriba hacia abajo. El dígito 0 representa una celda vacía (pista no definida).

Ejemplo:

```
1,Facil,003020600900305001001806400008102900700000008006708200002609500800203009005010
2,Media,200080300060070084030500209000105408000000000402706000301007040720040060004010003
```

jugadores.lfp

Este archivo contiene la información de los jugadores inscritos en el torneo.

Formato:

```
carnet,nombre,apellido,nivel
```

Donde:

- carnet: Identificador único del jugador (número entero)
- nombre: Nombre del jugador (cadena de texto)
- apellido: Apellido del jugador (cadena de texto)
- nivel: Categoría de experiencia (Principiante, Intermedio, Experto)

Ejemplo:

```
202011234,Diego,Fuentes,Intermedio
202012345,Ana,Morales,Experto
202013456,Carlos,Pérez,Principiante
```

intentos.lfp

Este archivo contiene los intentos de resolución realizados por los jugadores.

Formato:

```
carnet,id_sudoku,solucion,tiempo_segundos,fecha
```

Donde:

- carnet: Identificador del jugador que realizó el intento
- id_sudoku: Identificador del tablero resuelto
- solucion: cadena de 81 caracteres (dígitos 1-9) con la propuesta de solución del jugador, en el mismo orden de recorrido que el tablero original
- tiempo_segundos: Tiempo que tardó el jugador en resolverlo (número entero)
- fecha: Fecha en que se registró el intento (formato DD-MM-AAAA)

Ejemplo:

```
202011234,1,4839216579673458212518764935481329767295641381367982453726895148142537696954173
82,342,15-03-2026
202012345,1,4839216579673458212518764935481329767295641381367982453726895148142537696954173
82,198,15-03-2026
```

Mecánica de Validación (Reglas del Juego)

Para calificar un intento, el sistema debe:

- Reconstruir el tablero original y la solución propuesta como matrices de 9x9 a partir de las cadenas de 81 caracteres.
- Verificar que las celdas que ya tenían un valor fijo (distinto de 0) en el tablero original se mantengan sin cambios en la solución propuesta. Si una pista fue modificada, esa celda se marca como inválida.
- Validar, para cada una de las 9 filas, las 9 columnas y las 9 cajas de 3x3, que contengan los dígitos del 1 al 9 sin repetirse.
- Calcular el porcentaje de validez del intento: $(\text{cantidad de filas} + \text{columnas} + \text{cajas que cumplen la regla}) / 27 \times 100$.
- Considerar un intento como “resuelto correctamente” únicamente si el porcentaje de validez es 100% y todas las pistas originales se respetaron.

Funcionalidades del Sistema

El programa deberá permitir:

- Cargar los archivos de sudokus, jugadores e intentos desde el sistema
- Procesar la información utilizando el método `split(',')`

- Representar tableros, jugadores e intentos como objetos de clases (Programación Orientada a Objetos)
- Representar cada tablero e intento como una matriz de 9x9 (lista de listas)
- Validar cada intento según la mecánica de validación descrita
- Calcular métricas de desempeño (tiempo promedio, porcentaje de validez y tasa de éxito) sobre los intentos registrados
- Relacionar la información entre los tres archivos
- Generar reportes analíticos en formato HTML
- Mostrar un menú interactivo en consola

Reportes a Generar

El sistema deberá generar los siguientes reportes:

Reporte 1: Resumen por Sudoku

Muestra para cada tablero: identificador, dificultad declarada, cantidad de intentos recibidos, tiempo promedio de resolución y tasa de éxito (porcentaje de intentos resueltos correctamente).

Reporte 2: Rendimiento por Jugador

Muestra para cada jugador: nombre completo, carnet, nivel, cantidad de tableros intentados, porcentaje de validez promedio, tiempo promedio de resolución y cantidad de tableros resueltos perfectamente.

Reporte 3: Top 10 Mejores Tiempos

Lista ordenada de los 10 mejores tiempos entre los intentos resueltos correctamente (100% de validez), mostrando: posición, carnet, nombre completo, identificador del tablero, dificultad y tiempo.

Menú del Sistema

El programa deberá contar con un menú similar al siguiente:

```
=====
      TORNEO DE SUDOKU - NUMERIX
=====
1. Cargar archivo de sudokus
2. Cargar archivo de jugadores
3. Cargar archivo de intentos
4. Validar y calificar intentos
5. Generar Reporte: Resumen por Sudoku
6. Generar Reporte: Rendimiento por Jugador
7. Generar Reporte: Top 10 Mejores Tiempos
8. Salir

Seleccione una opción:
```

2.1 Descripción del problema a resolver

Una plataforma educativa organiza torneos de Sudoku y necesita automatizar la calificación de los intentos de sus jugadores, así como analizar el comportamiento del torneo: qué tableros resultan realmente más difíciles, quiénes son los jugadores más rápidos y precisos, y cómo varía el desempeño según el nivel de experiencia.

Para ello, la plataforma proporciona tres archivos de entrada:

- Un archivo con los tableros de Sudoku disponibles
- Un archivo con los jugadores inscritos
- Un archivo con los intentos de resolución registrados

El sistema deberá procesar estos archivos, validar cada intento contra las reglas del Sudoku, relacionar la información entre ellos, calcular métricas de desempeño y generar reportes analíticos que permitan visualizar de manera clara y precisa el desarrollo del torneo.

2.2 Alcance de la práctica

Obligatorio

- Uso del lenguaje Python (versión 3.x)
- Programación Orientada a Objetos (clases Tablero, Jugador e Intento, con sus atributos y métodos)
- Lectura de archivos .lfp mediante funciones nativas (open(), with, readlines())
- Separación de datos mediante el método split(',')
- Representación de tableros como matrices (listas de listas)
- Validación de filas, columnas y cajas de 3x3 según las reglas del Sudoku
- Uso de estructuras de control (if, while, for)
- Cálculo de métricas de desempeño (tiempo promedio, porcentaje de validez y tasa de éxito)
- Generación de reportes HTML
- Menú funcional en consola

Opcional

- Manejo de excepciones al leer archivos o tableros con formato incorrecto
- Implementación de un solucionador automático de Sudoku (backtracking) como validación adicional
- Gráficos estadísticos en los reportes HTML (mediante librerías JavaScript)

2.3 Requerimientos técnicos

La práctica debe desarrollarse haciendo uso de:

- Python como lenguaje de programación.
- Programación Orientada a Objetos (clases y objetos) para representar los tableros, jugadores e intentos del torneo.
- Lectura de archivos mediante funciones nativas del lenguaje (open(), with, readlines()).
- Separación y análisis de datos utilizando el método split(',') de las cadenas (str).
- Representación de tableros y soluciones como matrices (listas de listas) de 9x9.
- Uso de estructuras de control (condicionales, ciclos) y colecciones nativas (listas, diccionarios) para almacenar y procesar los registros.
- Generación de archivos HTML para reportes de resultados.
- Implementación de un menú basado en consola para la interacción con el usuario.

Los estudiantes deberán documentar su código y organizar el proyecto en un repositorio GitHub, con manual técnico y de usuario.

3. Entregables

Tipo	Descripción										
Repositorio en GitHub	Repositorio con el nombre LFP_<SECCIÓN>_2S2026, que contenga la carpeta del proyecto Practical con todo el código fuente en Python, así como un archivo README.md con las instrucciones de ejecución y ejemplos de uso. <table border="1"> <thead> <tr> <th>Sección</th><th>Usuario Tutor</th></tr> </thead> <tbody> <tr> <td>A</td><td>Adriluuu</td></tr> <tr> <td>B+</td><td>SamuelAguilar18</td></tr> <tr> <td>B-</td><td>JoseOre12</td></tr> <tr> <td>N</td><td>JorgeP3</td></tr> </tbody> </table>	Sección	Usuario Tutor	A	Adriluuu	B+	SamuelAguilar18	B-	JoseOre12	N	JorgeP3
Sección	Usuario Tutor										
A	Adriluuu										
B+	SamuelAguilar18										
B-	JoseOre12										
N	JorgeP3										
Código Fuente	Implementación completa, organizada y documentada según buenas prácticas de programación (PEP 8), utilizando clases y objetos (POO). El código debe incluir la lectura de archivos delimitados (split(',')), la validación matricial del Sudoku, el cálculo de métricas de desempeño, la generación de reportes en HTML y el manejo de excepciones.										
Manual Técnico	Documento en PDF o MD que describa la estructura del programa, las funciones o clases utilizadas, la lógica de validación matricial y los requerimientos técnicos. Debe incluir diagramas o pseudocódigo que expliquen la validación de filas, columnas y cajas.										
Manual de Usuario	Documento en PDF o MD con instrucciones claras para ejecutar el programa desde consola, cargar archivos, calificar intentos, generar reportes y navegar por el menú. Debe incluir ejemplos prácticos y capturas de pantalla del sistema en funcionamiento.										
Diagrama de Flujo	Representación gráfica del proceso general del sistema, incluyendo la lectura de archivos, la validación matricial del tablero, el cálculo de estadísticas y la generación de reportes.										
Informe de Desarrollo	Documento en PDF o MD que detalle la implementación de la práctica, los retos técnicos encontrados (especialmente en la validación de matrices) y las soluciones aplicadas. Debe incluir capturas del menú en ejecución, resultados de los reportes generados y conclusiones finales.										
Reportes Generados (HTML)	Conjunto de archivos .html que muestran los resultados solicitados: estadísticas por sudoku, rendimiento por jugador, top 10 mejores tiempos, sudokus más difíciles y análisis por nivel.										
Enlace de Entrega (UED)	Enlace oficial de entrega en la plataforma UEDI, apuntando al repositorio de GitHub con todos los materiales consolidados (código, documentación y reportes).										

4. Material de apoyo

Pressman, R. (2014). Ingeniería del Software: Un enfoque práctico. McGraw-Hill.

(Capítulos sobre estructuras de control, modularidad y buenas prácticas de desarrollo).

Sebesta, R. W. (2012). Conceptos de Lenguajes de Programación (9ª edición). Pearson Educación.

(Lectura recomendada para comprender la evolución, clasificación y fundamentos de los lenguajes de programación).

Downey, A. (2015). Think Python: How to Think Like a Computer Scientist (2ª edición). Green Tea Press.

(Capítulos sobre manejo de cadenas, lectura de archivos, listas anidadas y estructuras de control en Python).

Norvig, P. (2006). Solving Every Sudoku Puzzle. (Artículo de referencia sobre representación y validación algorítmica de tableros de Sudoku).

5. Recursos y herramientas a utilizar

Software:

- Python instalado en el equipo de trabajo (versión 3.x).
- Entorno de desarrollo (IDE) o editor de texto (PyCharm, Visual Studio Code, IDLE o equivalente).
- Consola del sistema operativo o terminal integrada del IDE para ejecutar y probar el programa.

Plataformas:

- GitHub: para el repositorio y control de versiones.
- UEDI: para la entrega oficial del proyecto y documentación.

Recursos de apoyo:

- Manuales de programación estructurada y fundamentos de diseño algorítmico.
- Documentación oficial de Python (docs.python.org).
- Artículos y guías sobre representación y validación de matrices.
- Recursos sobre diagramas de flujo y modelado de procesos secuenciales.

Lecturas recomendadas:

- Fundamentos de estructuras de control (condicionales y repetitivas) en Python.
- Manejo de listas anidadas (matrices) en Python.
- Manejo de archivos y excepciones en Python (open, try/except).
- Material introductorio sobre cálculo de porcentajes y promedios simples.

6. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Práctica	04-08-2026	18-08-2026



Calificación	21-08-2026	22-08-2026
--------------	------------	------------



7. Rúbrica de Calificación

7.1 Requisitos para optar a la calificación

Tema	Descripción	Cumple (Sí/No)
Cumplimiento de la tecnología establecida	El desarrollo debe haberse realizado en Python, aplicando Programación Orientada a Objetos, estructuras de control, manejo de cadenas, matrices y lectura de archivos.	
Uso de herramientas requeridas	El proyecto debe ejecutarse correctamente en PyCharm, Visual Studio Code, IDLE o equivalente, sin errores de ejecución.	
Gestión y entregas de la práctica	El repositorio en GitHub debe incluir el código fuente completo, el archivo README.md, y los documentos solicitados (manuales, reportes, diagramas).	
Documentación obligatoria	Debe entregarse el Manual Técnico, Manual de Usuario, Diagrama de Flujo y los reportes HTML generados, todos en formato PDF/HTML según corresponda.	
Pruebas y funcionalidad mínima	El sistema debe permitir la carga de los tres archivos .lfp, validar correctamente los intentos según las reglas del Sudoku y generar los 3 reportes solicitados en HTML.	

7.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
1. Habilidades		
Uso de estructuras de control (ciclos, condicionales)	1	
Manejo de cadenas y métodos (split y validaciones)	1	
Organización y legibilidad del código	1	
Entrega & Documentación (repo, manuales, diagramas)	1	
Interactividad del menú (funcionalidad básica)	1	
Pruebas y ejecución correcta	1	
Sub-Total Habilidades	6 pts	
2. Conocimiento		
Lectura y procesamiento de archivos (sudokus, jugadores, intentos)	2	
Validación matricial del Sudoku y cálculo de métricas de desempeño (promedio, porcentaje de validez, tasa de éxito)	4	



Generación de reportes en HTML (3 reportes completos y funcionales)	3	
Sub-Total Conocimiento	9 pts	
TOTAL GENERAL	15 pts	





7.3 Comentarios Generales



Detalle de la Calificación

No.	Criterio de evaluación	Punteo Máximo	Satisfactorio (100%–61%)	Necesita mejorar (60%–0%)	Punteo Obtenido
1.1	Uso de estructuras de control (ciclos, condicionales)	1	Utiliza correctamente ciclos y estructuras condicionales para controlar el flujo del programa y resolver los procesos planteados.	El uso de estructuras de control es limitado o incorrecto, generando errores o comportamientos inesperados.	
1.2	Manejo de cadenas y métodos (split y validaciones)	1	Implementa correctamente el método split(',') y realiza validaciones adecuadas sobre los datos extraídos.	El método split() no se usa correctamente o presenta errores en la separación y validación de los datos.	
1.3	Organización y legibilidad del código	1	El código está bien estructurado, con clases y objetos claros, indentación, comentarios y nombres de variables apropiados, siguiendo el estándar PEP 8.	El código carece de estructura o claridad, dificultando su lectura y mantenimiento.	
1.4	Entrega & Documentación (repo, manuales, diagramas)	1	Entrega completa del repositorio con manual técnico, manual de usuario y diagrama de flujo en el formato solicitado.	La entrega carece de alguno de los documentos requeridos o no sigue el formato establecido.	
1.5	Interactividad del menú (funcionalidad básica)	1	El menú permite al usuario navegar entre las opciones principales y ejecutar correctamente las acciones del sistema.	El menú presenta errores o no permite la ejecución fluida de las funcionalidades.	
1.6	Pruebas y ejecución correcta	1	El programa se ejecuta correctamente, mostrando resultados coherentes con los archivos de entrada.	El programa presenta errores o genera resultados incorrectos al ejecutar las pruebas.	
	Sub-total Habilidades	6 pts			
2.1	Lectura y procesamiento de archivos (sudokus.lfp, jugadores.lfp, intentos.lfp)	2	El sistema lee y procesa correctamente los tres archivos, separando los datos y almacenándolos en estructuras adecuadas.	El sistema no procesa correctamente los archivos o presenta errores en la lectura y separación de datos.	
2.2	Validación matricial del Sudoku y cálculo de métricas de desempeño (promedio, porcentaje de validez, tasa de éxito)	4	Implementa correctamente la validación de filas, columnas y cajas de 3x3, y calcula correctamente las métricas de desempeño requeridas sobre los intentos registrados.	La validación del tablero o el cálculo de métricas presentan errores, están incompletos o no se implementaron correctamente.	
2.3	Generación de reportes en HTML (3 reportes completos y funcionales)	3	Los 3 reportes HTML muestran la información requerida de forma ordenada, legible y con formato profesional. Incluyen todos los datos solicitados.	Los reportes presentan errores, información incompleta, formato incorrecto o faltan reportes.	
	Sub-total Conocimiento	9 pts			



	Total General	15 pts			
--	---------------	--------	--	--	--

